

Explications Techniques — IAM / PAM

Fonctionnement simple et détaillé de chaque mécanisme clé du projet

Table des matières

1. Face ID — Reconnaissance faciale
2. Les deux JWT — Keycloak vs Face Auth
3. Session SSH — xterm.js + WebSocket + Bastion
4. Session RDP — Guacamole + WebSocket + guacd
5. Session Base de données — Tunnel SSH + JDBC

1. Face ID — Reconnaissance faciale

L'idée simple

Au lieu de taper "alice123", ton visage **est** ton mot de passe. Le système transforme ton visage en 128 nombres, les stocke, et compare à chaque connexion.

La technologie : `face-api.js`

C'est une bibliothèque JavaScript qui fait tourner de l'intelligence artificielle **directement dans le navigateur** (TensorFlow.js). Elle charge 3 modèles de deep learning depuis un CDN :

Modèle	Ce qu'il fait
SSD MobileNet	Détecte où se trouve le visage dans l'image (rectangle de détection)
Face Landmark	Repère 68 points clés sur le visage (coins des yeux, bout du nez...)

Modèle	Ce qu'il fait
Face Recognition	Calcule le descripteur : vecteur de 128 nombres décimaux

Important : aucune image n'est jamais envoyée au serveur. Seul le tableau de 128 nombres part vers le backend.

Le descripteur facial — qu'est-ce que c'est ?

C'est une "empreinte mathématique" de ton visage. Un tableau de 128 valeurs décimales :

```
[0.12, -0.34, 0.87, 0.01, -0.56, 0.23, ..., 0.44]
  ↑      ↑      ↑      ↑
  128 valeurs qui décrivent uniquement TON visage
```

Deux photos du même visage → vecteurs très proches

Deux visages différents → vecteurs éloignés

La comparaison : distance euclidienne

Quand tu essaies de te connecter, le backend compare deux vecteurs avec cette formule :

$$\text{distance} = \sqrt{(a[0]-b[0])^2 + (a[1]-b[1])^2 + \dots + (a[127]-b[127])^2}$$

C'est exactement la distance entre deux points, mais dans un espace à 128 dimensions.

```
distance < 0.5 → même personne ✓ → connexion autorisée
distance > 0.5 → personne différente ✗ → connexion refusée
```

Les deux opérations

Enrollment (enregistrement de ton visage)

```
Tu ouvres /profile/face-verify
↓
Caméra s'active → flux vidéo en direct
↓
face-api.detectSingleFace(video)
  .withFaceLandmarks()
  .withFaceDescriptor()
↓
Float32Array[128] = [0.12, -0.34, ...]
↓
POST /api/user/face/enroll
{ "descriptor": [0.12, -0.34, ...] }
↓
Backend → stocke en base PostgreSQL
table: user_face_descriptors
{ username: "alice", descriptorJson: "[0.12, -0.34, ...]", isActive: true }
```

Login (connexion par visage)

```
Tu ouvres /login → saisit "alice" → caméra s'active
↓
face-api capture le descripteur → Float32Array[128]
↓
POST /api/public/face/login ← route PUBLIQUE (pas besoin de JWT)
{ "username": "alice", "descriptor": [0.08, -0.31, ...] }
↓
Backend FaceDescriptorService.verifyFace():
  → charge le descripteur stocké pour "alice"
  → calcule distance = 0.27 → < 0.5 → MATCH ✓
↓
FaceTokenService.generateToken("alice", ["user"], "tenant-bank-a")
  → génère un JWT signé HS256 (expliqué dans la section 2)
↓
Réponse: { token: "eyJhbGci...", username: "alice", roles: ["user"], ... }
↓
Angular → faceAuth.saveSession(resp) → localStorage
  → iam_face_token = "eyJhbGci..."
  → iam_face_profile = '{"username":"alice","roles":["user"],"tenantId":"tenant-bank-a"}'
↓
Redirect vers /user/my-requests
```

Les fichiers

Frontend

Fichier	Rôle
<code>face.service.ts</code>	Charge les modèles TensorFlow, capture le descripteur, appels API enroll/verify
<code>face-auth.service.ts</code>	Gère le JWT Face dans <code>localStorage</code> : save, get, clear, isAuthenticated
<code>login.component.ts</code>	Section Face ID de la page de connexion
<code>face-verify.component.ts</code>	Page <code>/profile/face-verify</code> pour tester/enroller

Backend

Fichier	Rôle
<code>FaceDescriptorController.java</code>	5 endpoints REST (enroll, verify, status, delete, public/login)
<code>FaceDescriptorService.java</code>	Logique : stockage, comparaison euclidienne
<code>FaceTokenService.java</code>	Génère le JWT HS256 après verification réussie
<code>FaceDescriptorRepository.java</code>	Accès base de données — table <code>user_face_descriptors</code>

Base de données

Table: `shared.user_face_descriptors`

<code>tenant_id</code>	<code>username</code>	<code>descriptor_json</code>	<code>is_active</code>
<code>tenant-bankA</code>	<code>alice</code>	<code>[0.12, -0.34, 0.87, 0.01, ..., 0.44]</code>	<code>true</code>

↑ 128 flottants sérialisés en JSON texte

JWT Keycloak		JWT Face Auth
		["tenant-bank-a"] ajouté manuellement
Durée	5 min (+ refresh token 30 jours)	24 heures (configurable)
Stockage	Géré par keycloak-angular (mémoire)	localStorage (iam_face_token)

Anatomie d'un JWT Keycloak (payload décodé)

```
{
  "sub": "4f8a2b1c-...",
  "iss": "http://localhost:8080/realms/iam-pam-realm",
  "preferred_username": "alice",
  "email": "alice@bank-a.com",
  "given_name": "Alice",
  "family_name": "Dupont",
  "realm_access": {
    "roles": ["user", "default-roles-iam-pam-realm"]
  },
  "groups": ["/tenant-bank-a"],
  "exp": 1746700300,
  "iat": 1746700000
}
```

Anatomie d'un JWT Face Auth (payload décodé)

```
{
  "sub": "alice",
  "iss": "face-auth",
  "preferred_username": "alice",
  "roles": ["user"],
  "tenantId": "tenant-bank-a",
  "groups": ["tenant-bank-a"],
  "exp": 1746786400,
  "iat": 1746700000
}
```

Différence notable : `roles` est une liste directe, et `iss` vaut `"face-auth"` et non une URL Keycloak.

Comment Spring Security distingue les deux ?

C'est le décodeur composite dans `SecurityConfig.java` :

```
// Décodeur Keycloak (RSA via JWK Set)
NimbusJwtDecoder keycloakDecoder = NimbusJwtDecoder
    .withJwkSetUri("http://localhost:8080/realms/iam-pam-realm/protocol/openid-connect/certs")
    .build();

// Décodeur Face Auth (HMAC avec secret partagé)
SecretKeySpec secretKey = new SecretKeySpec(
    faceJwtSecret.getBytes(), "HmacSHA256"
);
NimbusJwtDecoder faceDecoder = NimbusJwtDecoder.withSecretKey(secretKey).build();

// Décodeur composite – choisit selon l'émetteur
return token -> {
    String iss = parseIssuer(token); // lit le payload sans vérifier
    if ("face-auth".equals(iss)) {
        return faceDecoder.decode(token);    // vérifie avec HMAC
    }
    return keycloakDecoder.decode(token);    // vérifie avec RSA
};
```

Étape par étape :

1. Le JWT arrive dans le header `Authorization: Bearer eyJ...`
 2. Spring Security extrait le token
 3. Le décodeur composite parse le payload **sans vérifier** pour lire `iss`
 4. Si `iss == "face-auth"` → décodeur HMAC-256
 5. Sinon → décodeur RSA Keycloak
 6. La signature est vérifiée avec la bonne clé
 7. Si invalide → 401 Unauthorized
-

Comment les rôles sont extraits ?

```
private Collection<GrantedAuthority> extractAuthorities(Jwt jwt) {  
  
    // Face JWT : roles directement dans "roles"  
    if ("face-auth".equals(jwt.getClaimAsString("iss"))) {  
        List<String> roles = jwt.getClaimAsStringList("roles");  
        return roles.stream()  
            .map(r -> new SimpleGrantedAuthority("ROLE_" + r))  
            .collect(toList());  
        // ["user"] → [ROLE_user]  
    }  
  
    // Keycloak JWT : roles dans realm_access.roles  
    Map<String, Object> realmAccess = jwt.getClaimAsMap("realm_access");  
    List<String> roles = (List<String>) realmAccess.get("roles");  
    return roles.stream()  
        .map(r -> new SimpleGrantedAuthority("ROLE_" + r))  
        .collect(toList());  
    // ["user"] → [ROLE_user]  
}
```

RS256 vs HS256 — la différence technique

RS256 (Keycloak) — Asymétrique

```
Keycloak signe avec clé PRIVÉE RSA  
    ↓  
JWT contient la signature  
  
Backend vérifie avec clé PUBLIQUE RSA  
    (téléchargée depuis /.well-known/jwks.json)  
    ↓  
✓ si la signature correspond
```

Avantage : la clé publique peut être partagée librement. N'importe qui peut **vérifier** le token, mais seul Keycloak peut en **créer** un.

HS256 (Face Auth) — Symétrique

FaceTokenService signe avec SECRET partagé

↓

JWT contient la signature

Backend vérifie avec le MÊME SECRET

(stocké dans application-local.properties)

↓

✓ si la signature correspond

Avantage : plus simple à implémenter.

Inconvénient : quiconque connaît le secret peut créer un token valide → le secret doit rester **strictement privé**.

Cycle de vie des tokens

KEYCLOAK JWT

Durée : 5 minutes

Refresh : automatique par
keycloak-angular (transparent)

Stockage : mémoire JS

Révocation : via Keycloak

FACE AUTH JWT

Durée : 24 heures

Refresh : manuel via
POST /api/public/face/refresh

Stockage : localStorage

Révocation : clearToken() → localStorage.removeItem()

Côté Angular — le point d'entrée unique : AuthService

```
// Un seul endroit pour tout savoir sur l'identité
get isFaceAuth(): boolean {
    return !this.keycloak.isLoggedIn() && this.faceAuth.isAuthenticated();
}

get username(): string {
    if (isFaceAuth) return faceAuth.getUsername(); // lit localStorage
    return jwt.preferred_username; // lit Keycloak
}

get roles(): string[] {
    if (isFaceAuth) return faceAuth.getRoles(); // lit localStorage
    return keycloak.getUserRoles(true); // lit Keycloak
}

// Utilisé par le terminal SSH et le viewer RDP pour s'authentifier au WebSocket
async getToken(): Promise<string> {
    if (isFaceAuth) return faceAuth.getToken(); // depuis localStorage
    return keycloak.getToken(); // rafraîchit si besoin
}
```

Le reste de l'application (composants, interceptors, guards) ne sait **pas** quelle méthode est utilisée — il passe toujours par `AuthService`.

3. Session SSH — xterm.js + WebSocket + Bastion

L'idée simple

Tu veux ouvrir un terminal sur un serveur distant (`10.0.0.100`). Mais tu es dans un navigateur — pas de SSH natif. Voici comment le projet contourne ça :

```
Navigateur (xterm.js) ↔ WebSocket ↔ Spring Boot ↔ SSH ↔ Bastion VM ↔ SSH ↔ Serveur cible
```

Le backend joue le rôle d'un **relais** : il reçoit tes frappes clavier depuis le navigateur via WebSocket, les envoie au serveur SSH, et renvoie la réponse du serveur au navigateur.

Les technologies

Technologie	Rôle
xterm.js	Émulateur de terminal dans le navigateur (affiche les caractères, couleurs ANSI, curseur)
WebSocket	Canal bidirectionnel persistant entre Angular et Spring Boot
Apache SSHD	Client SSH en Java — s'exécute côté backend Spring Boot
Bastion VM	Serveur intermédiaire (192.168.112.138) — seul point d'entrée réseau
sshpas	Utilitaire Linux sur le bastion pour se connecter sans prompt de mot de passe

Pourquoi passer par un bastion ?

Sans bastion :

Navigateur → Internet → Serveur de production (port 22 exposé) ← DANGEREUX

Avec bastion :

Navigateur → Spring Boot → Bastion (seul serveur exposé) → Serveur de production

Les serveurs de production **ne sont jamais directement accessibles** de l'extérieur. Seul le bastion est joignable, et seulement par le backend avec une **clé SSH privée**.

Flux complet étape par étape

1. L'utilisateur clique "Lancer Terminal"

Angular route: /user/terminal/56
TerminalComponent se charge

2. xterm.js s'initialise dans le navigateur

```
this.term = new Terminal({
  cursorBlink: true,
  fontSize: 14,
  fontFamily: '"JetBrains Mono", monospace',
  theme: { background: '#0d1117', cursor: '#58a6ff', ... }
});
this.fitAddon = new FitAddon();
this.term.loadAddon(this.fitAddon);
this.term.open(containerRef.nativeElement); // attache au DOM
this.fitAddon.fit();                       // adapte la taille
```

3. Connexion WebSocket

```
const token = await this.auth.getToken(); // JWT Keycloak ou Face
const url = `ws://localhost:8081/ws/session/56?token=${token}`;
this.ws = new WebSocket(url);
```

Pourquoi le token dans l'URL ? Le navigateur ne peut pas envoyer de header HTTP custom (Authorization) lors de l'upgrade WebSocket. Le JWT passe donc en query parameter.

4. Spring Boot valide le JWT

```
// BastionTerminalHandler.afterConnectionEstablished()
String token = URI.getQueryParam("token");
jwtDecoder.decode(token); // lève JwtException si invalide → close(POLICY_VIOLATION)
Long requestId = extractFromPath("/ws/session/56") → 56L
bastionService.openSession(56L, wsSession);
```

5. Le backend ouvre la connexion SSH vers le bastion

```
// BastionService.openSession()
ClientSession bastionSession = sshClient
    .connect("bastion-agent", "192.168.112.138", 22)
    .verify(10s)
    .getSession();

bastionSession.addPublicKeyIdentity(loadKeyPair("classpath:bastion_key"));
bastionSession.auth().verify(15s); // authentication par clé RSA
```

La clé privée `bastion_key` (fichier RSA dans les ressources Spring) est utilisée pour s'authentifier sur le bastion **sans mot de passe**.

6. Un shell PTY s'ouvre sur le bastion

```
ChannelShell shell = bastionSession.createShellChannel();
shell.setUsePty(true);           // PTY = Pseudo Terminal – nécessaire pour les couleurs ANSI
shell.setPtyType("xterm-256color");

// Pipes I/O
PipedOutputStream toShell = new PipedOutputStream();    // Browser → Shell
PipedInputStream shellOutput = new PipedInputStream();   // Shell → Browser

shell.setIn(fromShellIn);
shell.setOut(toWs);
shell.setErr(toWs); // stderr aussi
shell.open().verify(10s);
```

7. Connexion depuis le bastion vers le serveur cible

```
String sshCmd = buildSshCommand("ubuntu", "10.0.0.100", 22, "s3cret");
// Si mot de passe : "sshpass -p 's3cret' ssh -o StrictHostKeyChecking=no -p 22 ubuntu@10.0.0.100"
// Si clé privée : "ssh -o StrictHostKeyChecking=no -p 22 ubuntu@10.0.0.100"

toShell.write((sshCmd + "\n").getBytes(UTF_8));
toShell.flush();
```

Le bastion exécute cette commande dans son shell → il se connecte SSH au serveur cible.

8. Thread de relay en arrière-plan

```
Thread relay = new Thread(() -> {
    byte[] buf = new byte[4096];
    int n;
    while (wsSession.isOpen() && (n = shellOutput.read(buf)) != -1) {
        wsSession.sendMessage(new TextMessage(new String(buf, 0, n, UTF_8)));
    }
});
relay.setDaemon(true);
relay.setName("bastion-relay-56");
relay.start();
```

Ce thread tourne en permanence et pompe la sortie SSH vers le WebSocket → s'affiche dans xterm.js.

9. Frappe clavier de l'utilisateur

```
Browser: l'utilisateur tape "ls -la" dans xterm.js
↓
xterm.js.onData(data) → ws.send("ls -la\n")
↓
WebSocket → Spring Boot
↓
BastionTerminalHandler.handleTextMessage(session, message)
↓
bastionService.sendInput(56, "ls -la\n")
↓
toShell.write("ls -la\n".getBytes(UTF_8))
↓
Shell SSH sur le bastion → STDIN du process SSH
↓
Serveur cible reçoit la commande → l'exécute → renvoie la sortie
↓
relayOutputToWs → wsSession.sendMessage(output)
↓
WebSocket → Angular → ws.onmessage → term.write(event.data)
↓
xterm.js affiche "total 48\ndrwxr-xr-x ..."
```

10. Audit — chaque frappe est enregistrée

```
public void sendInput(Long id, String input) {
    // ... écrit dans le shell ...
    String cleaned = input.replaceAll("[\\x00-\\x1F\\x7F]", "").trim(); // supprime les codes
    if (!cleaned.isEmpty()) {
        auditService.log(COMMAND_EXECUTED, username, tenantId, resourceName, id, "CMD: " + cleaned);
    }
}
```

Résultat dans la base : chaque commande tapée est visible dans les logs d'audit.

11. Fermeture de session

```
L'utilisateur ferme l'onglet ou clique "Déconnecter"
↓
WebSocket.onclose → BastionTerminalHandler.afterConnectionClosed()
↓
bastionService.closeSession(56)
↓
shell.close(true) + bastionSession.close(true)
↓
auditService.log(SESSION_ENDED, ...)
↓
activeSessions.remove(56) ← nettoyage mémoire
```

Les fichiers

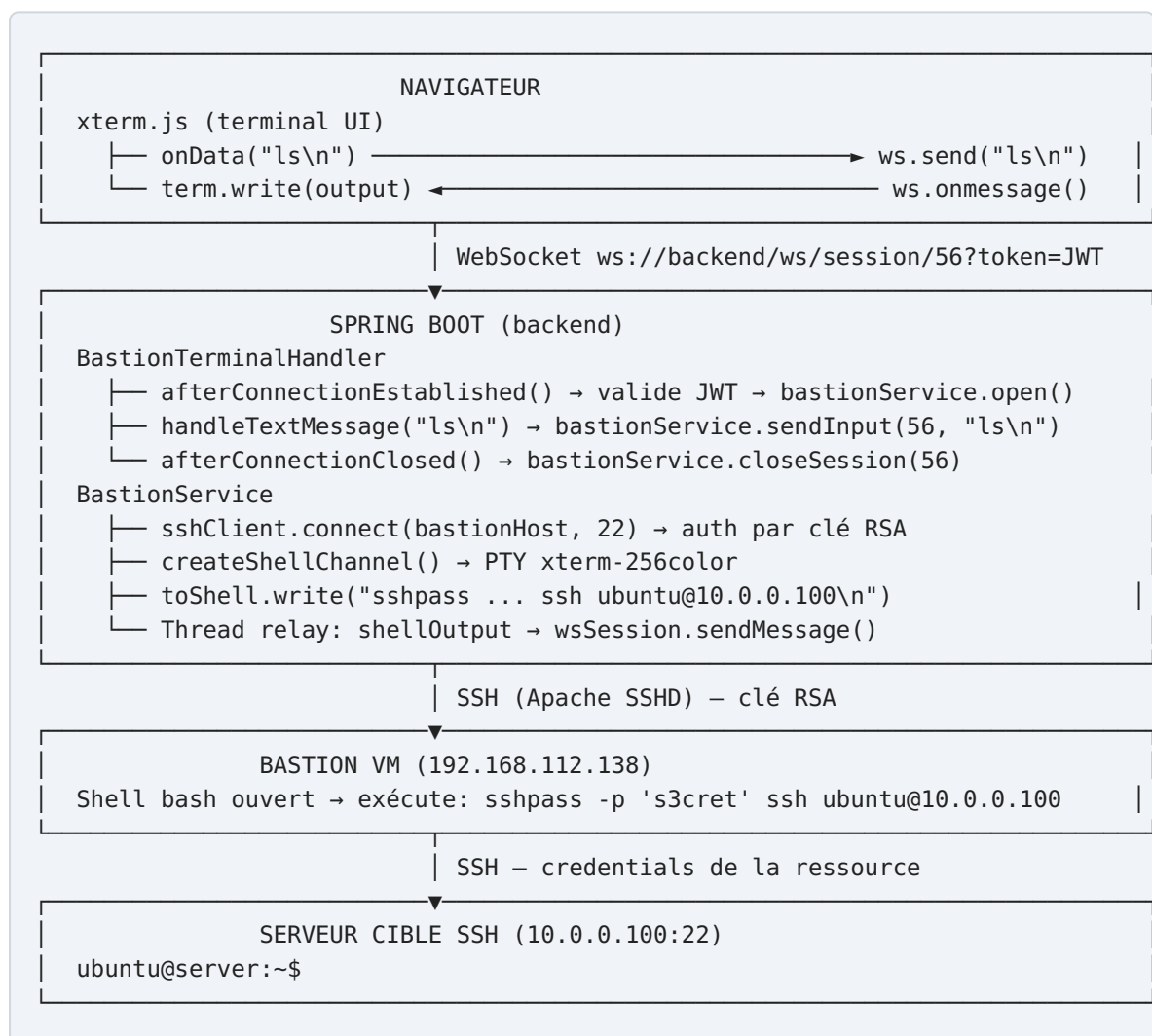
Frontend

Fichier	Rôle
<code>terminal.component.ts</code>	Initialise xterm.js, ouvre WebSocket, relaie les I/O
<code>terminal.component.html</code>	Contient le <code><div #terminalContainer></code> où xterm.js se monte
<code>terminal.component.scss</code>	Styles du terminal (fond noir, plein écran)

Backend

Fichier	Rôle
<code>BastionTerminalHandler.java</code>	Handler WebSocket — valide le JWT, relaie les messages vers <code>BastionService</code>
<code>BastionService.java</code>	Ouvre la session SSH, gère les pipes I/O, relay output, audit
<code>WebSocketConfig.java</code>	Enregistre le handler sur <code>/ws/session/{requestId}</code>
<code>bastion_key</code> (resources)	Clé privée RSA pour s'authentifier sur le bastion

Schéma récapitulatif



4. Session RDP — Guacamole + WebSocket + guacd

L'idée simple

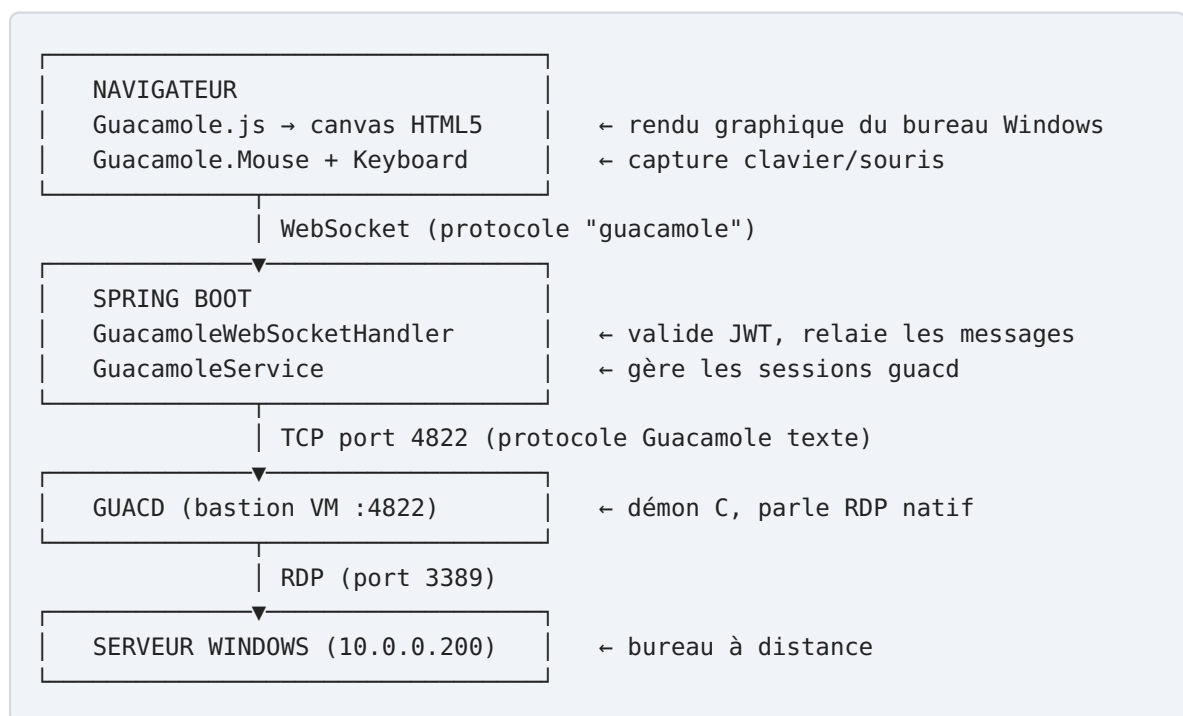
RDP c'est le protocole de bureau à distance de Microsoft (Remote Desktop Protocol). Le problème : un navigateur web ne peut pas parler RDP nativement. La solution s'appelle **Apache Guacamole**.

Navigateur (canvas HTML5) ↔ WebSocket ↔ Spring Boot ↔ TCP:4822 ↔ guacd ↔ RDP ↔ Windows

Les technologies

Technologie	Rôle
guacd	Démon C qui parle RDP natif vers le serveur Windows
Apache Guacamole Client (Java)	Bibliothèque Java qui parle au guacd via TCP:4822
Protocole Guacamole	Format texte propriétaire pour transmettre écran/clavier/souris
guacamole-common-js	Bibliothèque JS qui reçoit les instructions Guacamole et dessine sur un <code><canvas></code>
WebSocket	Canal entre Angular et Spring Boot

L'architecture en couches



Le protocole Guacamole — qu'est-ce que c'est ?

C'est un format texte propriétaire pour encoder les événements graphiques, clavier et souris. Exemple de messages :

```
5.image,3.png,1.0,1.0,10.png-data...; ← envoyer une image (portion d'écran)
3.key,1.65432,1.1; ← touche pressée
4.size,4.1280,3.720; ← redimensionner le bureau
5.mouse,3.250,3.300,1.4; ← position + état du clic souris
```

Chaque instruction est préfixée par sa longueur. Spring Boot relaie ces messages tels quels entre le WebSocket et guacd.

Le sous-protocole WebSocket `guacamole`

Lors du handshake WebSocket, le navigateur envoie :

```
Sec-WebSocket-Protocol: guacamole
```

Le serveur **doit** répondre avec ce même sous-protocole, sinon guacamole-common-js ferme la connexion immédiatement.

```
// WebSocketConfig.java
.setHandshakeHandler(new DefaultHandshakeHandler() {
    @Override
    protected String selectProtocol(List<String> requestedProtocols, ...) {
        if (requestedProtocols.contains("guacamole")) return "guacamole";
        return super.selectProtocol(requestedProtocols, ...);
    }
})
```

Flux complet étape par étape

1. L'utilisateur clique "Ouvrir Bureau RDP"

```
Angular route: /user/rdp/57
RdpViewerComponent se charge
```

2. Guacamole.js s'initialise dans le navigateur

```
// Guacamole est chargé depuis un CDN via index.html
declare const Guacamole: any;

const token = await this.auth.getToken();
const wsUrl = `ws://localhost:8081/ws/rdp/57`;

// Crée le tunnel WebSocket
this.tunnel = new Guacamole.WebSocketTunnel(wsUrl);

// Crée le client Guacamole
this.client = new Guacamole.Client(this.tunnel);

// Récupère le canvas et l'ajoute au DOM
const display = this.client.getDisplay().getElement();
this.containerRef.nativeElement.appendChild(display);

// Configure clavier + souris
this.keyboard = new Guacamole.Keyboard(document);
this.keyboard.onkeydown = (keysym) => this.client.sendKeyEvent(1, keysym);
this.keyboard.onkeyup = (keysym) => this.client.sendKeyEvent(0, keysym);

this.mouse = new Guacamole.Mouse(this.containerRef.nativeElement);
this.mouse.onmousemove = (state) => this.client.sendMouseState(state);
this.mouse.onmousedown = (state) => this.client.sendMouseState(state);
this.mouse.onmouseup = (state) => this.client.sendMouseState(state);

// Lance la connexion – génère ws://backend/ws/rdp/57?token=JWT
this.client.connect(`token=${token}`);
```

3. WebSocket upgrade avec négociation du sous-protocole

```
Browser → Spring Boot
GET /ws/rdp/57?token=eyJ...
Upgrade: websocket
Sec-WebSocket-Protocol: guacamole

Spring → Browser
HTTP/1.1 101 Switching Protocols
Sec-WebSocket-Protocol: guacamole ← OBLIGATOIRE
```

4. Spring Boot valide le JWT

```
// GuacamoleWebSocketHandler.afterConnectionEstablished()
String token = URI.getQueryParam("token");
jwtDecoder.decode(token); // valide le JWT (Keycloak ou Face)
Long requestId = extractFromPath("/ws/rdp/57") -> 57L
sessionRequestMap.put(wsSession.getId(), 57L);

// Setup guacd dans un thread séparé (bloque ~800ms, ne doit pas bloquer Tomcat)
Thread setup = new Thread(() -> guacamoleService.openSession(57L, wsSession));
setup.setDaemon(true);
setup.setName("guac-setup-57");
setup.start();
```

Pourquoi un thread séparé ? Le `ConfiguredGuacamoleSocket` effectue le handshake avec guacd qui prend ~800ms. Si ce code s'exécute dans le thread WebSocket de Tomcat, la connexion peut être interrompue.

5. Spring Boot se connecte à guacd

```
// GuacamoleService.openSession()
Resource resource = accessRequest.getResource();

// Configuration RDP
GuacamoleConfiguration config = new GuacamoleConfiguration();
config.setProtocol("rdp");
config.setParameter("hostname", "10.0.0.200");
config.setParameter("port", "3389");
config.setParameter("username", "Administrator");
config.setParameter("password", "WinPass!");
config.setParameter("width", "1280");
config.setParameter("height", "720");
config.setParameter("color-depth", "16");
config.setParameter("ignore-cert", "true");

// Connexion TCP vers guacd sur le bastion
InetGuacamoleSocket socket = new InetGuacamoleSocket("192.168.112.138", 4822);

// Handshake guacd : envoie la config, guacd ouvre une connexion RDP vers Windows
ConfiguredGuacamoleSocket configuredSocket = new ConfiguredGuacamoleSocket(socket, config);

// Tunnel Guacamole = wrapper pour lire/écrire les instructions
GuacamoleTunnel tunnel = new SimpleGuacamoleTunnel(configuredSocket);
```

6. guacd se connecte au serveur Windows en RDP

guacd reçoit la config: protocol=rdp, hostname=10.0.0.200, port=3389, ...
guacd → RDP handshake → Serveur Windows
guacd reçoit des frames graphiques RDP
guacd les encode en instructions Guacamole

7. Thread de relay guacd → WebSocket

```
private void relayToWs(GuacamoleTunnel tunnel, WebSocketSession wsSession, Long requestId) {
    while (wsSession.isOpen() && tunnel.isOpen()) {
        char[] chars = tunnel.acquireReader().read(); // lit les instructions guacd
        tunnel.releaseReader();
        String instruction = new String(chars);
        synchronized (wsSession) {
            wsSession.sendMessage(new TextMessage(instruction)); // envoie au navigateur
        }
    }
}
```

guacamole-common-js reçoit ces instructions et **dessine** le bureau Windows dans le canvas HTML5.

8. Action utilisateur (clic souris, frappe clavier)

```
L'utilisateur clique à position (500, 300)
↓
Guacamole.Mouse → client.sendMouseState(state)
↓
guacamole-common-js encode : "5.mouse,3.500,3.300,1.1;"
↓
ws.send("5.mouse,3.500,3.300,1.1;")
↓
GuacamoleWebSocketHandler.handleTextMessage()
↓
guacamoleService.sendInstruction(57, "5.mouse,3.500,3.300,1.1;")
↓
tunnel.acquireWriter().write(instruction.toCharArray())
↓
guacd → RDP mouse event → Windows
↓
Windows déplace le curseur → met à jour l'écran
↓
guacd encode le rafraîchissement → instruction "5.image,..."
↓
relay → wsSession → canvas HTML5 mis à jour
```

9. Redimensionnement

```
this.resizeObserver = new ResizeObserver(() => this.sendSize());
// ...
private sendSize(): void {
    const w = el.clientWidth || 1280;
    const h = el.clientHeight || 720;
    this.client.sendSize(w, h);
}
```

Quand l'utilisateur redimensionne la fenêtre → `sendSize` → guacd → RDP → Windows adapte la résolution.

10. Fermeture

```
L'utilisateur clique "Déconnecter"
↓
this.client.disconnect()
↓
GuacamoleWebSocketHandler.afterConnectionClosed()
↓
guacamoleService.closeSession(57)
↓
tunnel.close()
↓
auditService.log(SESSION_ENDED, ...)
↓
activeSessions.remove(57)
```

Les fichiers

Frontend

Fichier	Rôle
<code>rdp-viewer.component.ts</code>	Init Guacamole.js, gère clavier/souris/resize, connect/disconnect
<code>rdp-viewer.component.html</code>	Contient <code><div #displayContainer></code> où le canvas Guacamole se monte

Backend

Fichier	Rôle
<code>GuacamoleWebSocketHandler.java</code>	

Fichier	Rôle
	Handler WebSocket — valide JWT, délègue à <code>GuacamoleService</code>
<code>GuacamoleService.java</code>	Gère les sessions guacd : open, relay, send, close, audit
<code>WebSocketConfig.java</code>	Enregistre le handler sur <code>/ws/rdp/{requestId}</code> avec sous-protocole <code>guacamole</code>

Infrastructure

Composant	Rôle
<code>guacd</code> (port 4822 sur bastion VM)	Démon C — traduit RDP ↔ protocole Guacamole
<code>guacamole-common-js</code> (CDN)	Bibliothèque JS cliente

5. Session Base de données — Tunnel SSH + JDBC

L'idée simple

Tu veux exécuter des requêtes SQL sur une base de données de production (`10.0.0.50:5432`). Mais cette base n'est pas accessible directement depuis Internet — elle n'accepte que les connexions depuis le bastion.

La solution : créer un **tunnel SSH** qui redirige un port local (disons `41523`) vers la base via le bastion.

DbViewer (navigateur) ↔ REST HTTP ↔ Spring Boot ↔ JDBC sur port local ↔ Tunnel SSH ↔ Bastion

Note : contrairement au SSH et au RDP, la session DB utilise **HTTP classique** (pas de WebSocket) pour les requêtes SQL.

Les technologies

Technologie	Rôle
Apache SSHD	Client SSH Java — crée le tunnel (port forwarding)
SSH Local Port Forwarding	Redirige localhost:PORT_LOCAL → bastion → db_host:db_port
JDBC	API Java standard pour parler à une base de données
HttpClient Angular	Requêtes REST classiques pour les opérations SQL
DbViewerComponent	Interface utilisateur : éditeur SQL + arbre de schéma

Le principe du SSH Port Forwarding

C'est une fonctionnalité SSH standard. Elle crée un "tuyau" :

```
Spring Boot écoute localhost:41523
  ↓ (tout ce qui arrive sur ce port)
  ↓ passe dans le tunnel SSH vers le bastion
  ↓
Bastion renvoie vers 10.0.0.50:5432
  ↓
PostgreSQL répond
  ↓ chemin inverse
Spring Boot reçoit la réponse sur localhost:41523
```

Du point de vue de JDBC, il croit se connecter à `localhost:41523` — il ne sait pas qu'il y a un tunnel SSH derrière.

Flux complet étape par étape

1. L'utilisateur clique "Ouvrir la base de données"

```
Angular route: /user/db/58
DbViewerComponent.ngOnInit()
  ↓
this.startSession()
  ↓
POST /api/pam/db/start/58 {}
Authorization: Bearer <JWT>
```

2. Spring Boot démarre la session DB

```
// DbTunnelService.startSession(58)

// Vérification de la demande d'accès
AccessRequest request = findByIdWithResource(58);
// isActive() → true (APPROVED, pas expiré)

Resource resource = request.getResource();
String dbHost = "10.0.0.50";
int    dbPort = 5432;           // PostgreSQL
String dbUser = "app_user";
String dbPass = "secret123";
String dbName = "production_db"; // stocké dans resource.description
String dbType = detectDbType(5432) → "postgresql"
```

3. Connexion SSH vers le bastion

```
ClientSession bastionSession = sshClient
    .connect("bastion-agent", "192.168.112.138", 22)
    .verify(10s)
    .getSession();

bastionSession.addPublicKeyIdentity(loadKeyPair("bastion_key"));
bastionSession.auth().verify(15s);
```

4. Port forwarding SSH

```
// Trouver un port libre sur localhost
int localPort = findFreePort(); // ex: 41523

// Déclarer: localhost:41523 → via bastion → 10.0.0.50:5432
SshdSocketAddress local  = new SshdSocketAddress("localhost", 41523);
SshdSocketAddress remote = new SshdSocketAddress("10.0.0.50", 5432);
bastionSession.startLocalPortForwarding(local, remote);

Thread.sleep(300); // attendre que le port soit prêt
```

Après cette instruction, **n'importe quel process sur la machine** qui se connecte à `localhost:41523` sera automatiquement redirigé vers `10.0.0.50:5432` via le tunnel SSH.

5. Connexion JDBC via le tunnel

```
// JDBC croit se connecter à localhost:41523
// En réalité il passe par le tunnel SSH vers PostgreSQL
String url = "jdbc:postgresql://localhost:41523/production_db?connectTimeout=10";

Properties props = new Properties();
props.setProperty("user", "app_user");
props.setProperty("password", "secret123");

Connection jdbcConn = DriverManager.getConnection(url, props);
```

6. Une `sessionId` unique est retournée au frontend

```
String sessionId = UUID.randomUUID().toString();
// ex: "a3f7b2c1-1234-5678-abcd-ef0123456789"

sessions.put(sessionId, new ActiveDbSession(
    sessionId, 58, bastionSession, jdbcConn, 41523, "postgresql", "production_db", ...
));

return new DbSessionInfo(sessionId, "postgresql", "production_db");
```

```
← { sessionId: "a3f7b2c1-...", dbType: "postgresql", dbName: "production_db" }
```

Angular stocke le `sessionId` en mémoire (variable du composant). Toutes les requêtes suivantes l'utiliseront.

7. Chargement du schéma de la base

```
GET /api/pam/db/schema/a3f7b2c1-...
↓
DbTunnelService.fetchSchema("a3f7b2c1-...")
↓
session.jdbcConn().getMetaData()
→ meta.getSchemas() → ["public", "app", "audit"]
→ meta.getTables(null, "public", "%", ["TABLE", "VIEW"])
→ ["users", "orders", "products", ...]
→ meta.getColumns(null, "public", "users", "%")
→ ["id INTEGER", "name VARCHAR", "email VARCHAR", ...]
↓
← [
  { name: "public", tables: [
    { name: "users", columns: ["id INTEGER", "name VARCHAR", ...] },
    { name: "orders", columns: ["id INTEGER", "user_id INTEGER", ...] }
  ]},
  { name: "app", tables: [...] }
]
```

Angular affiche un arbre de schémas/tables dans le panneau gauche du DbViewer.

8. L'utilisateur exécute une requête SQL

```
L'utilisateur tape: SELECT * FROM users WHERE id < 10  
Clique "Exécuter"
```

```
↓  
POST /api/pam/db/query/a3f7b2c1-...  
{ "sql": "SELECT * FROM users WHERE id < 10" }
```

```
// DbTunnelService.executeQuery("a3f7b2c1-...", "SELECT * FROM users WHERE id < 10")  
  
Statement stmt = jdbcConn.createStatement();  
stmt.setMaxRows(500);           // limite de sécurité  
stmt.setQueryTimeout(30);       // timeout 30 secondes  
  
ResultSet rs = stmt.executeQuery("SELECT * FROM users WHERE id < 10");  
ResultSetMetaData meta = rs.getMetaData(); // noms des colonnes  
  
// Lit toutes les lignes  
List<String> columns = ["id", "name", "email", "created_at"];  
List<List<Object>> rows = [  
    ["1", "Alice", "alice@bank.com", "2026-01-15"],  
    ["2", "Bob", "bob@bank.com", "2026-01-16"],  
    ...  
];  
  
auditService.log(COMMAND_EXECUTED, username, tenantId, resourceName, 58,  
    "SQL: SELECT * FROM users WHERE id < 10");
```

```
← {  
  type: "SELECT",  
  columns: ["id", "name", "email", "created_at"],  
  rows: [["1","Alice","alice@bank.com","2026-01-15"], ...],  
  rowCount: 9,  
  executionMs: 23  
}
```

Angular affiche le résultat dans un tableau Material.

9. Requête DML (INSERT, UPDATE, DELETE)

```
// Si la requête ne commence pas par SELECT/SHOW/EXPLAIN/WITH  
int affected = stmt.executeUpdate("UPDATE users SET active=false WHERE id=5");  
return { type: "UPDATE", rowsAffected: 1, executionMs: 12 }
```

10. Fermeture de la session

```
L'utilisateur clique "Déconnecter" (ou ferme l'onglet)
↓
ngOnDestroy() → DELETE /api/pam/db/stop/a3f7b2c1-...
↓
DbTunnelService.endSession("a3f7b2c1-...")
↓
jdbcConn.close()           ← ferme la connexion JDBC
bastionSession.close()     ← ferme le tunnel SSH (et donc le port forwarding)
auditService.log(SESSION_ENDED, ...)
sessions.remove("a3f7b2c1-...")
```

Quand le tunnel SSH se ferme, `localhost:41523` n'existe plus — il est impossible de continuer à interroger la base.

Détection automatique du type de base

```
private String detectDbType(int port) {
    return switch (port) {
        case 5432, 5433 -> "postgresql";
        case 3306, 3307 -> "mysql";
        case 27017      -> "mongodb";
        case 1521       -> "oracle";
        default        -> "postgresql";
    };
}
```

Et l'URL JDBC s'adapte :

```
String url = switch (dbType) {
    case "mysql" -> "jdbc:mysql://localhost:" + localPort + "/" + dbName + "?useSSL=false";
    default      -> "jdbc:postgresql://localhost:" + localPort + "/" + dbName;
};
```

Les fichiers

Frontend

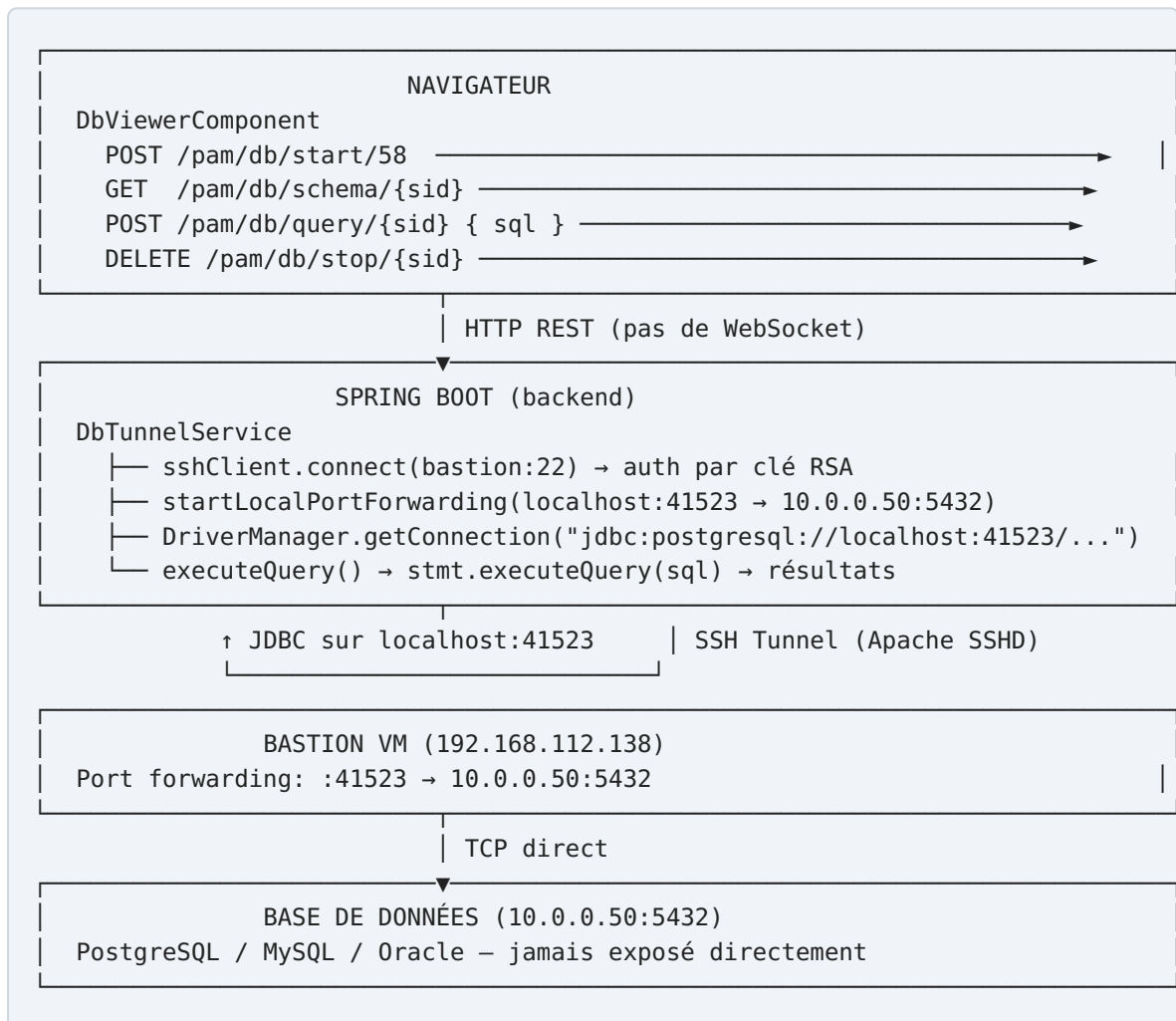
Fichier	Rôle
<code>db-viewer.component.ts</code>	Start/stop session, load schema, run query, affichage résultats

Fichier	Rôle
db-viewer.component.html	Éditeur SQL + arbre de schéma + tableau de résultats

Backend

Fichier	Rôle
DbTunnelService.java	Tunnel SSH, port forwarding, JDBC, executeQuery, fetchSchema
DbController.java	Endpoints REST : /pam/db/start/{id} , /pam/db/query/{sid} , /pam/db/schema/{sid} , /pam/db/stop/{sid}

Schéma récapitulatif



Différences clés entre SSH, RDP et DB

	Session SSH	Session RDP	Session DB
Transport	WebSocket	WebSocket	HTTP REST
Protocole temps réel	Texte brut (ANSI)	Protocole Guacamole	N/A (requête/réponse)
Ce qui transite	Frappes clavier + sortie terminal	Instructions graphiques (images, events)	Requêtes SQL + résultats JSON
Rendu	xterm.js (canvas terminal)	canvas HTML5 (bureau Windows)	Tableau Angular Material
Tunnel	SSH → shell → sshpass ssh cible	TCP:4822 → guacd → RDP	SSH Port Forwarding → JDBC
Audit	Chaque commande (COMMAND_EXECUTED)	Ouverture/ fermeture session	Chaque requête SQL (COMMAND_EXECUTED)

Fin du document — IAM/PAM Explications Techniques